



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DSA0204-Computer Vision with OpenCV

Name: K. Archana

Reg No: 192524209

1.Read an image in python and Convert an Image to Grayscale

AIM:

To convert an image to grayscale image.

PROGRAM:

```
import cv2  
image_path = r"C:\\Users\\Yaswini Reddy\\Downloads\\cv.jpg"  
image = cv2.imread(image_path)  
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  
cv2.imshow("Original Image", image)  
cv2.imshow("Grayscale Image", gray_image)
```

INPUT:



OUTPUT:



2.Read an image in python and Convert an Image to Blur using Gaussian Blur.

AIM:

To convert and image to gaussian blur

PROGRAM:

```
import cv2  
  
image_path = r"C:\Users\Yaswini Reddy\Downloads\cv.jpg"  
image = cv2.imread(image_path)  
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  
blurred_image = cv2.GaussianBlur(image, (7, 7), 0)  
cv2.imshow("Original Image", image)  
cv2.imshow("Grayscale Image", gray_image)  
cv2.imshow("Blurred Image", blurred_image)
```

INPUT:



OUTPUT:



3.Read an image in python and Convert an Image to show outline using Canny function.

AIM:

TO perform basic Image Handling and processing operations on the image

PROGRAM:

```
import cv2  
  
image_path = r"C:\Users\Yaswini Reddy\Downloads\cv.jpg"  
image = cv2.imread(image_path)  
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  
edges = cv2.Canny(gray, 100, 200)  
cv2.imshow("Original Image", image)  
cv2.imshow("Canny Edges", edges)
```

INPUT:



OUTPUT:



4. Read an image in python and Dilate an Image using Dilate function.

AIM:

To perform dilate an image using Dilate function.

PROGRAM:

```
import cv2
import numpy as np
image_path = r"C:\Users\Yaswini Reddy\Downloads\download.png"
image = cv2.imread(image_path)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
kernel = np.ones((5, 5), np.uint8)
dilated = cv2.dilate(binary, kernel, iterations=1)
cv2.imshow("Original Image", image)
cv2.imshow("Dilated Image", dilated)
```

INPUT:



OUTPUT:



5. Read an image in python and Erode an Image using erode function.

AIM:

To perform erode image using erode function.

PROGRAM:

```
import cv2
import numpy as np
image_path = r"C:\Users\Yaswini Reddy\Downloads\download.png"
image = cv2.imread(image_path)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
kernel = np.ones((5, 5), np.uint8)
eroded = cv2.erode(binary, kernel, iterations=1)
cv2.imshow("Original Image", image)
cv2.imshow("Eroded Image", eroded)
```

INPUT:



OUTPUT:



6.Read captured video in python and display the video, in slow motion and in fast motion.

AIM:

To read a captured video in slow and fast motion.

PROGRAM:

```
import cv2

video_path = r"C:\Users\Yaswini Reddy\Downloads\WhatsApp Video 2025-05-27 at 14.41.32_025d3595.mp4"

cap = cv2.VideoCapture(video_path)

while True:

    ret, frame = cap.read()

    if not ret:

        break

    cv2.imshow("Normal Speed", frame)

    cv2.waitKey(30)

    cv2.imshow("Slow Motion", frame)

    cv2.waitKey(100)

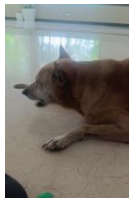
    cv2.imshow("Fast Motion", frame)

    cv2.waitKey(10)

    if cv2.waitKey(1) & 0xFF == ord('q'):

        break
```

INPUT:



OUTPUT:



8. Scaling an image to its bigger and smaller sizes.

AIM:

To perform scaling of an image to bigger and smaller sizes.

PROGRAM:

```
import cv2

image_path = r"C:\Users\Yaswini Reddy\Downloads\download.png"

image = cv2.imread(image_path)

bigger = cv2.resize(image, None, fx=2, fy=2, interpolation=cv2.INTER_LINEAR)

smaller = cv2.resize(image, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)

cv2.imshow("Original Image", image)

cv2.imshow("Scaled Up Image", bigger)

cv2.imshow("Scaled Down Image", smaller)
```

INPUT:



OUTPUT:



9. Perform Rotation of an image to clockwise and counter clockwise direction.

AIM:

To Perform Rotation of an image to clockwise and counter clockwise direction.

PROGRAM:

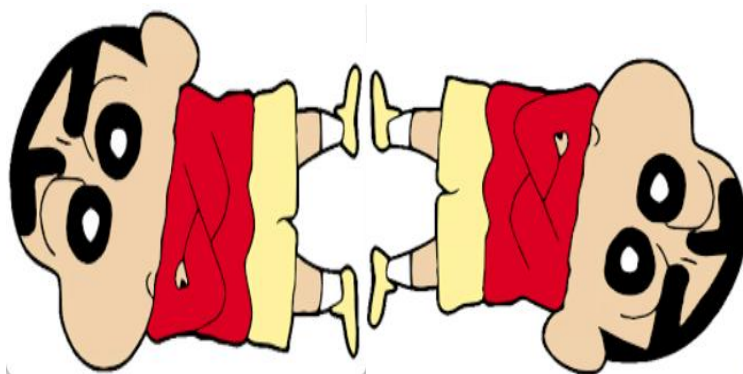
```
import cv2

image_path = r"C:\Users\Yaswini Reddy\Downloads\download.png"
image = cv2.imread(image_path)
clockwise = cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE)
counter_clockwise = cv2.rotate(image, cv2.ROTATE_90_COUNTERCLOCKWISE)
cv2.imshow("Original Image", image)
cv2.imshow("Clockwise Rotation", clockwise)
cv2.imshow("Counter-Clockwise Rotation", counter_clockwise)
```

INPUT:



OUTPUT:



10. Perform moving of an image from one place to another.

AIM:

To perform moving of an image from one place to another.

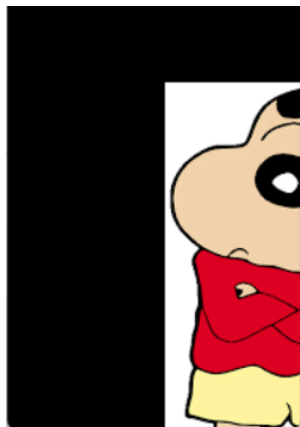
PROGRAM:

```
import cv2
import numpy as np
image_path = r"C:\Users\Yaswini Reddy\Downloads\download.png"
image = cv2.imread(image_path)
height, width = image.shape[:2]
matrix = np.float32([[1, 0, 100], [0, 1, 50]])
moved = cv2.warpAffine(image, matrix, (width, height))
cv2.imshow("Original Image", image)
cv2.imshow("Moved Image", moved)
```

INPUT:



OUTPUT:



11. Perform Affine Transformation on the image.

AIM:

To perform Affine Transformation on the image.

PROGRAM:

```
import cv2
import numpy as np
image_path = r"C:\Users\Yaswini Reddy\Downloads\download.png"
image = cv2.imread(image_path)
rows, cols = image.shape[:2]
pts1 = np.float32([[50, 50], [200, 50], [50, 200]])
pts2 = np.float32([[10, 100], [200, 50], [100, 250]])
matrix = cv2.getAffineTransform(pts1, pts2)
affine_transformed = cv2.warpAffine(image, matrix, (cols, rows))
cv2.imshow("Original Image", image)
cv2.imshow("Affine Transformed Image", affine_transformed)
```

INPUT:



OUTPUT:



12. Perform Perspective Transformation on the image.

AIM:

To perform Perspective Transformation on the image.

PROGRAM:

```
import cv2
import numpy as np
image_path = r"C:\Users\Yaswini Reddy\Downloads\download.png"
image = cv2.imread(image_path)
rows, cols = image.shape[:2]
pts1 = np.float32([[50, 50], [200, 50], [50, 200], [200, 200]])
pts2 = np.float32([[0, 0], [300, 0], [0, 300], [300, 300]])
matrix = cv2.getPerspectiveTransform(pts1, pts2)
perspective_transformed = cv2.warpPerspective(image, matrix, (300, 300))
cv2.imshow("Original Image", image)
cv2.imshow("Perspective Transformed Image", perspective_transformed)
```

INPUT:



OUTPUT:



14. Perform transformation using Homography matrix.

AIM:

To perform transformation using Homography matrix.

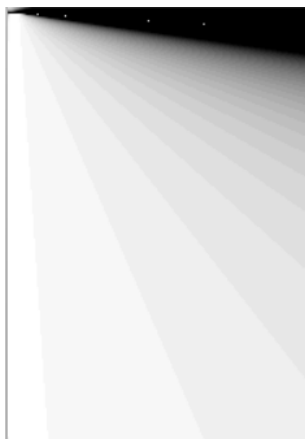
PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")
h, w = img.shape[:2]
H = np.array([[10, 20, 0.10],
              [30, 10, 5.0],
              [10, 3, 0.1]])
warped = cv2.warpPerspective(img, H, (w, h))
cv2.imshow("Original", img)
cv2.imshow("transformed", warped)
```

INPUT:



OUTPUT:



15. Perform transformation using Direct Linear Transformation.

AIM:

To Perform transformation using Direct Linear Transformation.

PROGRAM:

```
import cv2
import numpy as np
image_path = r"C:\Users\Yaswini Reddy\Downloads\download.png"
image = cv2.imread(image_path)
pts_src = np.float32([[50, 50], [200, 50], [50, 200], [200, 200]])
pts_dst = np.float32([[30, 70], [220, 60], [70, 230], [250, 240]])
H, _ = cv2.findHomography(pts_src, pts_dst, method=0)
dlt_transformed = cv2.warpPerspective(image, H, (image.shape[1],
image.shape[0]))
cv2.imshow("Original", image)
cv2.imshow("DLT Transformed", dlt_transformed)
```

INPUT:



OUTPUT:



16. Perform Edge detection using canny method.

AIM:

To perform Edge detection using canny method.

PROGRAM:

```
import cv2  
  
import numpy as np  
  
img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")  
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
edges = cv2.Canny(gray, 100, 200)  
cv2.imshow("Original Image", img)  
cv2.imshow("Canny Edges", edges)
```

INPUT:



OUTPUT:



17. Perform Edge detection using Sobel Matrix along X axis.

AIM:

To Perform Edge detection using Sobel Matrix along X axis.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobelx = cv2.convertScaleAbs(sobelx)
cv2.imshow("Original Image", img)
cv2.imshow("Sobel Edge - X axis", sobelx)
```

INPUT:



OUTPUT:



18. Perform Edge detection using Sobel Matrix along Y axis

AIM:

To Perform Edge detection using Sobel Matrix along Y axis.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobely = cv2.convertScaleAbs(sobely)
cv2.imshow("Original Image", img)
cv2.imshow("Sobel Edge - Y axis", sobely)
```

INPUT:



OUTPUT:



19. Perform Edge detection using Sobel Matrix along XY axis.

AIM:

To Perform Edge detection using Sobel Matrix along XY axis.

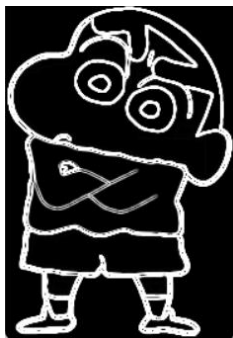
PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = cv2.magnitude(sobel_x, sobel_y)
edges_xy = cv2.convertScaleAbs(mag)
cv2.imshow("Original", img)
cv2.imshow("Sobel XY Edges", edges_xy)
```

INPUT:



OUTPUT:



20. Perform Sharpening of Image using Laplacian mask with negative center coefficient.

AIM:

To Perform Sharpening of Image using Laplacian mask with negative center coefficient.

PROGRAM:

```
import cv2

import numpy as np

img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")

kernel = np.array([[0, -1, 0],
                  [-1, 4, -1],
                  [0, -1, 0]])

sharpened = cv2.filter2D(img, -1, kernel)

cv2.imshow("Laplacian Sharpening", sharpened)
```

INPUT:



OUTPUT:



21. Perform Sharpening of Image using Laplacian mask implemented with an extension of diagonal neighbours.

1	1	1
1	-8	1
1	1	1

AIM:

To Perform Sharpening of Image using Laplacian mask implemented with an extension of diagonal neighbours.

PROGRAM:

```
import cv2

import numpy as np

img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")

kernel = np.array([[ -1, -1, -1],
                   [ -1,  8, -1],
                   [ -1, -1, -1]])

sharpened = cv2.filter2D(img, -1, kernel)

cv2.imshow("Laplacian with Diagonals", sharpened)
```

INPUT:



OUTPUT:



22. Perform Sharpening of Image using Laplacian mask with positive center coefficient.

Mask of Laplacian + addition

$$\begin{aligned}g(x, y) &= f(x, y) - [f(x+1, y) + f(x-1, y) \\&\quad + f(x, y+1) + f(x, y-1) + 4f(x, y)] \\&= 5f(x, y) - [f(x+1, y) + f(x-1, y) \\&\quad + f(x, y+1) + f(x, y-1)]\end{aligned}$$

0	-1	0
-1	5	-1
0	-1	0

AIM:

To Perform Sharpening of Image using Laplacian mask with positive center coefficient.

PROGRAM:

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")
kernel = np.array([[0, 1, 0],
                   [1, -4, 1],
                   [0, 1, 0]])
sharpened = cv2.filter2D(img, -1, kernel)
cv2.imshow("Laplacian Positive Center", sharpened)
```

INPUT:



OUTPUT:



23. Perform Sharpening of Image using unsharp masking.

Unsharp masking

$$f_s(x, y) = f(x, y) - \bar{f}(x, y)$$

sharpened image = original image – blurred image

AIM:

To Perform Sharpening of Image using unsharp masking.

PROGRAM:

```
import cv2
import numpy as np

img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")
blur = cv2.GaussianBlur(img, (5,5), 0)
mask = cv2.subtract(img, blur)
sharpened = cv2.add(img, mask)
cv2.imshow("Unsharp Masking", sharpened)
```

INPUT:



OUTPUT:



24. Perform Sharpening of Image using High-Boost Masks.

0	-1	0	-1	-1	-1
-1	$A + 4$	-1	-1	$A + 8$	-1
0	-1	0	-1	-1	-1

AIM:

To Perform Sharpening of Image using High-Boost Masks.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Yaswini Reddy\Downloads\download.png")
blur = cv2.GaussianBlur(img, (5,5), 0)
A = 1.5
high_boost = cv2.addWeighted(img, A, blur, -1, 0)
cv2.imshow("High-Boost Filter", high_boost)
```

INPUT:



OUTPUT:



25. Perform Sharpening of Image using Gradient masking.

-1	-2	-1	-1	0	1
0	0	0	-2	0	2
1	2	1	-1	0	1

AIM:

To Perform Sharpening of Image using Gradient masking.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg")
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
gradient = cv2.magnitude(sobel_x, sobel_y)
gradient = cv2.convertScaleAbs(gradient)
sharpened = cv2.add(img, gradient)
cv2.imshow("Gradient Masking", sharpened)
```

INPUT:



OUTPUT:



26. Insert water marking to the image using OpenCV.

AIM:

To Insert water marking to the image using OpenCV.

PROGRAM:

```
import cv2  
  
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg")  
cv2.putText(img, "Watermark", (20, img.shape[0] - 20),  
            cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)  
cv2.imshow("Watermarked", img)
```

INPUT:



OUTPUT:



27. Do Cropping, Copying and pasting image inside another image using OpenCV.

AIM:

To Do Cropping, Copying and pasting image inside another image using OpenCV.

PROGRAM:

```
import cv2  
  
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg")  
crop = img[10:40, 10:40]  
img[50:80, 50:80] = crop  
cv2.imshow("Crop and Paste", img)
```

INPUT:



OUTPUT:



28. Find the boundary of the image using Convolution kernel for the given image.

AIM:

To Find the boundary of the image using Convolution kernel for the given image.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg")
kernel = np.array([[0, -1, 0],
                  [-1, 4, -1],
                  [0, -1, 0]])
boundary = cv2.filter2D(img, -1, kernel)
cv2.imshow("Boundary Detected", boundary)
```

INPUT:



OUTPUT:



29. Morphological operations based on OpenCV using Erosion technique.

AIM:

To Morphological operations based on OpenCV using Erosion technique.

PROGRAM:

```
import cv2
import numpy as np
image = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg")
kernel = np.ones((5, 5), np.uint8)
eroded_image = cv2.erode(image, kernel, iterations=1)
cv2.imshow('Eroded Image', eroded_image)
```

INPUT:



OUTPUT:



30. Morphological operations based on OpenCV using Dilation technique.

AIM:

To Morphological operations based on OpenCV using Dilation technique.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg", 0)
kernel = np.ones((5,5), np.uint8)
dilated = cv2.dilate(img, kernel, iterations=1)
cv2.imshow('Dilated', dilated)
```

INPUT:



OUTPUT:



31. Morphological operations based on OpenCV using Opening technique.

AIM:

To Morphological operations based on OpenCV using Opening technique.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg", 0)
kernel = np.ones((5,5), np.uint8)
opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
cv2.imshow('Opening', opening)
```

INPUT:



OUTPUT:



32. Morphological operations based on OpenCV using Closing technique.

AIM:

To Morphological operations based on OpenCV using Closing technique.

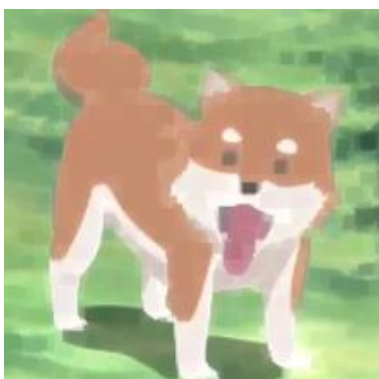
PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg")
kernel = np.ones((5,5), np.uint8)
closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
cv2.imshow('Closing', closing)
```

INPUT:



OUTPUT:



33. Morphological operations based on OpenCV using Morphological Gradient technique.

AIM:

To Morphological operations based on OpenCV using Morphological Gradient technique.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg")
kernel = np.ones((5,5), np.uint8)
morph_gradient = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)
cv2.imshow('Morphological Gradient', morph_gradient)
```

INPUT:



OUTPUT:



34. Morphological operations based on OpenCV using Top hat technique.

AIM:

To Morphological operations based on OpenCV using Top hat technique.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg")
kernel = np.ones((5,5), np.uint8)
tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)
cv2.imshow('Top Hat', tophat)
```

INPUT:



OUTPUT:



35. Morphological operations based on OpenCV using Black hat technique.

AIM:

To Morphological operations based on OpenCV using Black hat technique.

PROGRAM:

```
import cv2
import numpy as np
img = cv2.imread(r"C:\Users\Aishwarya\Downloads\cv.jpg")
kernel = np.ones((5,5), np.uint8)
blackhat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)
cv2.imshow('Black Hat', blackhat)
```

INPUT:



OUTPUT:



36. Recognise watch from the given image by general Object recognition using OpenCV.

AIM:

To Recognise watch from the given image by general Object recognition using OpenCV.

PROGRAM:

```
import cv2

scene_path = r'C:\Users\Ayyappan Studio\Documents\OneDrive\Pictures\exp2.jpg'
scene_image = cv2.imread(scene_path, 0)

template_path = r'C:\Users\Ayyappan Studio\Downloads\watch_template.jpg'
template = cv2.imread(template_path, 0)

if scene_image is None:
    print("Error loading scene image.")
    exit()

if template is None:
    print("Error loading template image.")
    exit()

w, h = template.shape[::-1]

result = cv2.matchTemplate(scene_image, template, cv2.TM_CCOEFF_NORMED)
min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(result)

top_left = max_loc
bottom_right = (top_left[0] + w, top_left[1] + h)

output_image = cv2.cvtColor(scene_image, cv2.COLOR_GRAY2BGR)
cv2.rectangle(output_image, top_left, bottom_right, (0, 255, 0), 2)

output_path = r'C:\Users\Ayyappan Studio\Documents\OneDrive\Pictures\watch_detected.jpg'
cv2.imwrite(output_path, output_image)

print(f"✅ Output saved to: {output_path}")
```

INPUT:



OUTPUT:



37. Using Opencv play Video in Reverse mode.

AIM:

To Using Opencv play Video in Reverse mode.

PROGRAM:

```
import cv2

video_path = r"C:\Users\Ayyappan Studio\Downloads\istockphoto-1676192640-640_adpp_is.mp4"

cap = cv2.VideoCapture(video_path)

frames = []

while True:

    ret, frame = cap.read()

    if not ret:

        break

    frames.append(frame)

cap.release()

for frame in reversed(frames):

    cv2.imshow('Reversed Video Playback', frame)
```

```
if cv2.waitKey(30) & 0xFF == ord('q'): # Press Q to quit early  
    break
```

INPUT:



OUTPUT:



38. Face Detection using Opencv.

AIM:

To Face Detection using Opencv.

PROGRAM:

```
import cv2

face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')

image_path = r"C:\Users\Ayyappan Studio\Documents\OneDrive\Pictures\exp2.jpg"
image = cv2.imread(image_path)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
cv2.imshow("Detected Faces", image)
```

INPUT:



OUTPUT:



40. Rectangular shape.

AIM:

To draw Rectangular shape and extract object.

PROGRAM:

```
import cv2
import numpy as np

# Load your image from the given path
img = cv2.imread(r'C:\Users\Ayyappan
Studio\Documents\OneDrive\Pictures\exp2.jpg')
if img is None:
    print("Error: Image not found or path is incorrect.")
    exit()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)

# Edge detection
edges = cv2.Canny(blur, 50, 150)

# Find contours
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

for i, cnt in enumerate(contours):
    approx = cv2.approxPolyDP(cnt, 0.02 * cv2.arcLength(cnt, True), True)
    if len(approx) == 4: # Detecting rectangular shape
        x, y, w, h = cv2.boundingRect(approx)
```

```
cv2.rectangle(img, (x,y), (x+w, y+h), (0, 255, 0), 2)
```

```
# Extract rectangular object and show in a separate window
```

```
extracted_obj = img[y:y+h, x:x+w]
```

```
cv2.imshow(f'Extracted Object {i+1}', extracted_obj)
```

```
cv2.imshow('Rectangular Shapes Detected', img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

INPUT:



OUTPUT:

